PROGETTAZIONE DI SOFTWARE SICURO

Lezione 6

I principi del design by contract e i contratti nel SW Object-Oriented

Elvinia Riccobene

Università degli Studi di Milano - SSRI

Bibliografia

Meyer, Bertrand

- Object-Oriented Software Construction, Prentice Hall, 1988

Inventò il linguaggio di programmazione EIFFEL

- la torre Eiffel come esempio:
  - costruita rispettando tempi e budget
  - a partire da "componenti"
  - costruita come cosa temporanea, invece c'è tutt'oggi
- EIFFEL Software ed EIFFEL Studio
- DbC: Design by contract

Design by contract

Cosa è un contratto?

Un accordo tra cliente (client) e fornitore (supplier) che:

- lega le due (o più) parti: fornitore e cliente
- è esplicito (scritto)
- specifica gli *obblighi* e i *benefici* delle due parti
- normalmente *mappa* gli obblighi di una parte come benefici dell'altra parte
- non contiene clausole nascoste: gli obblighi sono quelli dichiarati

Tipico dei rapporti tra persone

Esempio: contratto di costruzione

	Obblighi	Vantaggi
Cliente	Mettere a disposizione 1 ettaro di terreno edificabile e un tot di soldi	Ricevere un edificio di 3 piani entro tot mesi
Fornitore	Costruire un palazzo di 3 piani entro tot mesi	Non deve fare nulla se non riceve i soldi o se il terreno non è adatto

Esempio: contratto di viaggio

	Obblighi	Vantaggi
Cliente	Essere all'aeroporto di Santa Barbara almeno 5 minuti prima dell'orario di partenza previsto. Porta solo bagagli accettabili. Paga il prezzo del biglietto.	Raggiungi Chicago.
Fornitore	Porta il cliente a Chicago	Non è necessario trasportare passeggeri in ritardo, che hanno bagagli inaccettabili o non hanno pagato il prezzo del biglietto.

Contratti nel software

Oggetto del contratto: il software

- in OO è un insieme di classi

Cliente: chi usa/invoca il programma (i.e., classe)

Fornitore: chi scrive il programma (i.e., classe)

Il contratto di ogni classe dovrebbe essere:

- esplicito
- parte dello stesso elemento software

Linguaggi per contratti

Il linguaggio Eiffel è stato costruito seguendo i principi del DbC:

- il linguaggio di programmazione contiene il linguaggio per scrivere contratti

Nel Corso useremo Java come linguaggio di codifica e vedremo l'uso di asserzioni e JML per scrivere contratti

PRE e POST condizioni

In un contratto si può:

- **definire cosa ogni metodo richiede**
  - obbligo per il cliente
  - precondizioni
- **cosa fornisce il metodo**
  - obbligo per il fornitore
  - postcondizioni

Precondizioni e postcondizioni possono essere *espresse*:

- usando la logica
- usando la sintassi del linguaggio di programmazione

Esempio di un Array

Operazione `insert(element)` in un Array

- no_elements: numero di elementi memorizzati
- size: dimensione del vettore

Precondition

- l'array non è pieno:

```
no_elements < size
```

Postcondition

- l'elemento `element` è nell'array
- l'array contiene un elemento in più:

```
no_elements' = no_elements +1
```

  - no_elements' indica il valore dopo l'operazione

Perchè le precondizioni?

Un metodo deve gestire ogni possibile input?

- In generale **NO**
- Precondizioni **deboli** (TRUE significa nessuna precondizione); tutte le complicazione sono gestite dalla routine
- Precondizioni **forti** (FALSE significa che non può essere chiamato)

La scelta di precondizioni è una scelta di progetto; non c'e' una regola assoluta: però è meglio scrivere metodi semplici che soddisfino un contratto ben definito che un metodo che "cerca" di gestire tutte le situazioni possibili.

Come garantire le precondizioni

Il cliente deve garantire una precondizione prima di invocare il metodo. Come fare?

Supponiamo sia *pre* la precondizione per un metodo **`m`** di un oggetto **`x`**:

- `pre` potrebbe consistere nel verificare un campo `a` di **`x`**, o il valore ritornato dal chiamare un metodo `m1`
  - es. `x.a >0 o x.m1() == 1`,…

1. il cliente dovrà controllare l'invocazione del metodo `m` di `x` in questo modo:

```
if (pre) x.m()
else {/* special treatment */}
```

2. oppure essere sicuro che **`pre`** vale prima della chiamata di `m` tramite l'uso dei contratti

Come garantire le postcondizioni

Il fornitore deve garantire le postcondizioni nell'implementazione del metodo

- il cliente potrà assumere che le post condizioni siano vere dopo l’invocazione del metodo

Come funziona un contratto

Cliente: controlla le precondizioni prima di invocare un metodo ma assume (non controlla) che le postcondizioni valgono

Fornitore: assume (non controlla) che le precondizioni valgono e garantisce le postcondizioni

L'uso del contratto evita che il controllo venga fatto sia dal cliente che dal fornitore

Pre e Post condizioni: test

Supponi di essere un laureato in Informatica e sul mercatino cerchi lavoro per fare il programma A (con {P} A {Q}, P precondizioni e Q postcondizioni). Sei un po' sfaticato.

- **Preferisci P forte o debole?**
- **Q forte o debole?**

  P forte e Q debole rendono A più facile da scrivere
- **Due offerte speciali:**

  1. {False} A {...}

  2. {...} A {True}

  Quale scegli?

  nel primo caso non devi scrivere A
  nel secondo caso puoi implementare A come vuoi

Proprietà interne di una classe

Possiamo specificare nel contratto anche una proprietà che vale sempre per tutte le istanze

INVARIANTE

L'invariante è vero dopo la creazione dell'oggetto e dopo ogni operazione

Esempio per l'Array:

il numero di elementi è sempre compreso tra 0 e la dimensione massima:

```
0 <= no_elements <= size
```

L'invariante definisce un obbligo ulteriore

- l'implemetazione di ogni metodo non deve violare l'invariante
- è una precondizione ulteriore: chi implementa il metodo sa che l'invariante vale

Contratti come documenti

I contratti possono essere inseriti anche prima di fornire una vera implementazione del metodo

Documentazione delle classi

- dalle definizioni dei contratti si possono estrarre in modo automatico precondizioni, post condizioni ed invarianti che documentano cosa fa la classe

Ruolo delle eccezioni nel contratto

Le eccezioni si sollevano solo quando il contratto è violato

La violazione di un contratto è la manifestazione di un DIFETTO ("BUG")

- precondition (colpa del client)
- postcondition o invariant (colpa del fornitore)

Le eccezioni sono casi di violazione e NON casi particolari o alternativi

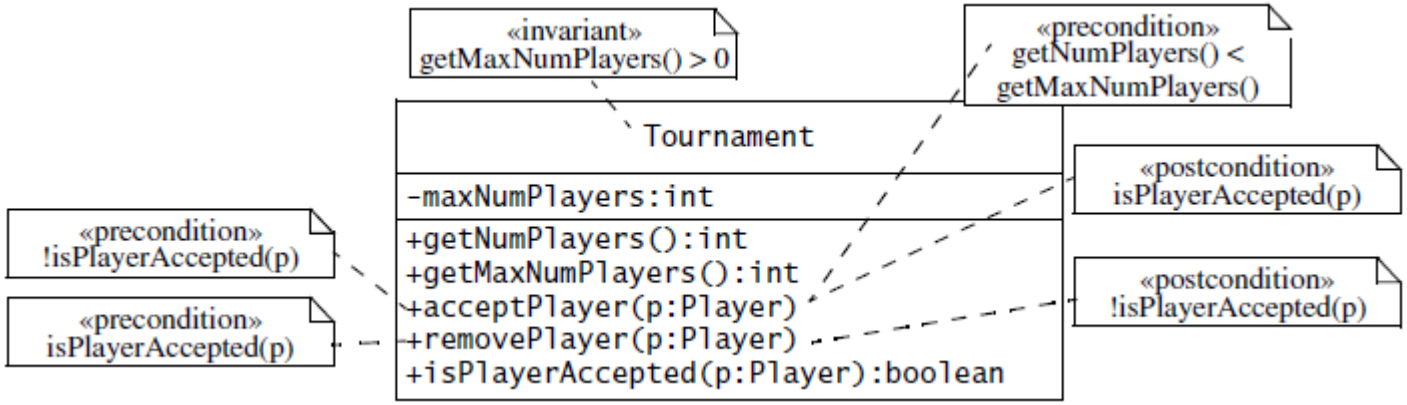
- Per casi particolari o alternativi si usano le strutture di controllo (es. "if the sum is negative, report an error...")

Esempio di contratto

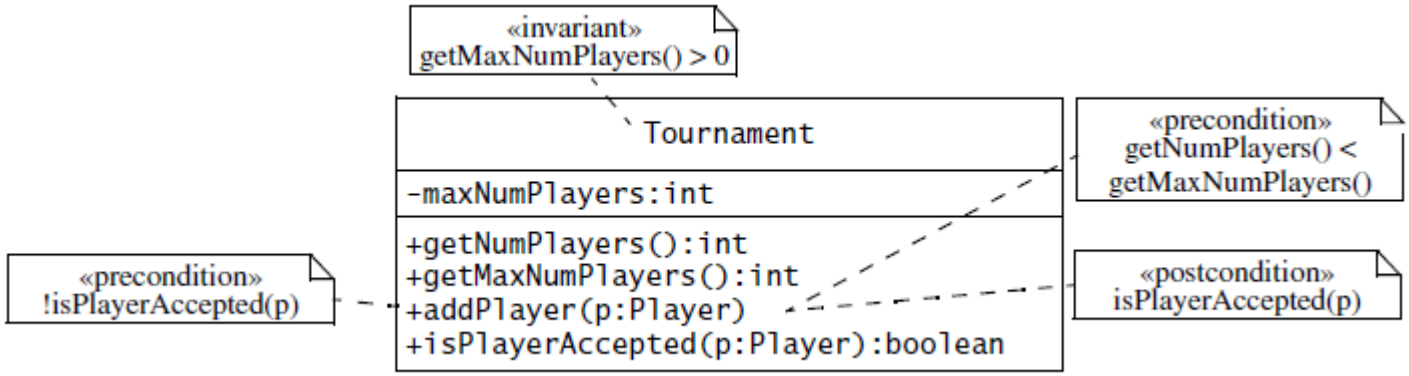
```java
/** A Tournament is a series of Matches among a set of Players
* which ends with a single winner. The Game and TournamentStyle of a
* Tournament is determined by the League in which the Tournament is
* played.
*/
public class Tournament {

    /** The maximum number of players is positive at all times.
     * @invariant maxNumPlayers > 0
     */
    private int maxNumPlayers;

    /** The players List contains references to Players who are
     *  are registered with the Tournament.
     */
    private List players;

    /* Constructors omitted */

    /** Returns the current number of players in the tournam
     */
    public int getNumPlayers() {…}

    /** Returns the maximum number of players in the tournam
     */
    public int getMaxNumPlayers() {…}

    /** The acceptPlayer() operation assumes that the specified player
     * has not been accepted in the Tournament yet.
     * @pre !isPlayerAccepted(p)
     * @pre getNumPlayers() < maxNumPlayers
     * @post isPlayerAccepted(p)
     * @post getNumPlayers() = self@pre.getNumPlayers() + 1
     */
    public void acceptPlayer (Player p) {…}

    /** The removePlayer() operation assumes that the specified player
     * is currently in the Tournament.
     * @pre isPlayerAccepted(p)
     * @post !isPlayerAccepted(p)
     * @post getNumPlayers() = self@pre.getNumPlayers() - 1
     */
    public void removePlayer(Player p) {…}

    /* Other methods omitted */
}
```

Contratti specificati come commenti e annotazioni Java

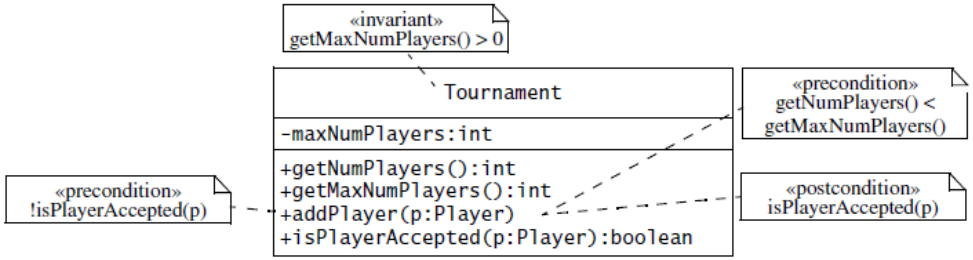
Esempio di contratto



Focus sul metodo `addPlayer`, che rinomina `acceptPlayer`

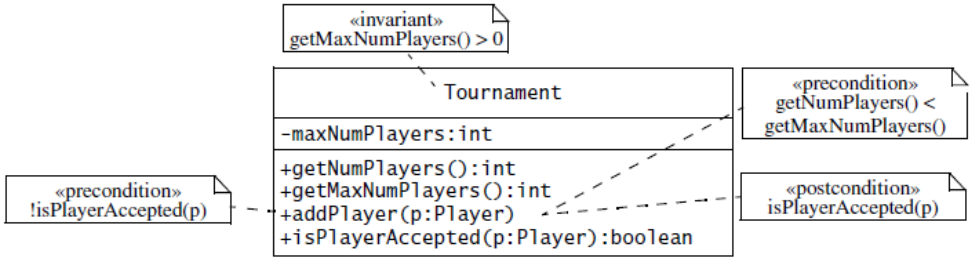


Eccezioni per le precondizioni




```
public class Tournament {
//...
    private List players;

    public void addPlayer(Player p)
        throws KnownPlayer, TooManyPlayers, UnknownPlayer,
            IllegalNumPlayers, IllegalMaxNumPlayers
    {
        // check precondition!isPlayerAccepted(p)
        if (isPlayerAccepted(p)) {
            throw new KnownPlayer(p);
        }
        // check precondition getNumPlayers() < maxNumPlayers
        if (getNumPlayers() == getMaxNumPlayers()) {
            throw new TooManyPlayers(getNumPlayers());
        }
        // save values for postconditions
        int pre_getNumPlayers = getNumPlayers();

        // accomplish the real work
        players.add(p);
        p.addTournament(this);
```

Eccezioni per le postcondizioni




```
        // accomplish the real work
        players.add(p);
        p.addTournament(this);

        // check post condition isPlayerAccepted(p)
        if (!isPlayerAccepted(p)) {
            throw new UnknownPlayer(p);
        }
        // check post condition getNumPlayers() = @pre.getNumPlayers() + 1
        if (getNumPlayers() != pre_getNumPlayers + 1) {
            throw new IllegalNumPlayers(getNumPlayers());
        }
        // check invariant maxNumPlayers > 0
        if (getMaxNumPlayers() <= 0) {
            throw new IllegalMaxNumPlayers(getMaxNumPlayers());
        }
    }
    //...
}
```

Usabilità delle eccezioni

Se mappassimo ogni contratto seguendo i passaggi precedenti, ci assicureremo che tutte le precondizioni, le postcondizioni e gli invarianti sono controllate per ogni chiamata del metodo e che le violazioni vengono rilevate all'interno di una chiamata di metodo.

Anche se questo approccio porta allo sviluppo di un software robusto, non è realistico

Alcuni sistemi

Eiffel – linguaggio OO simile a Simula

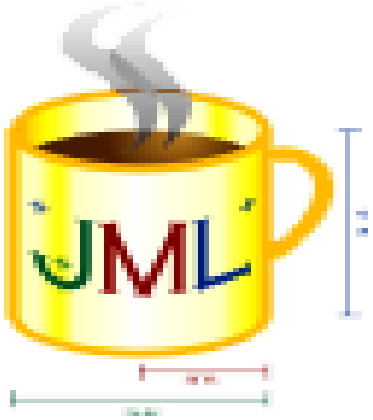
GNU Nana: per C e C++

iContract (Java)

(Java assertion in jdk 1.4)
http://java.sun.com/j2se/1.4/docs/guide/lang/assert.html

Jass: Java Assertions: http://semantik.informatik.uni-oldenburg.de/~jass/

JML: The Java Modeling Language (JML)



JAVA MODELING LANGUAGE

DbC con JML per Software Java

Vedremo come specificare le pre, post condizioni e invarianti in programmi Java usando JML

JML è un insieme di tool e di tecniche per il design by contract in Java ed altro

Sintassi simile per tutti i tool dbc e per tutti i linguaggi OO

Benefici di JML

- **Le specifiche JML forniscono una documentazione esplicita del contratto**
- **Scrivendo le specifiche in JML si rendono chiare le assunzioni fatte a livello di progetto**
  - che andranno considerate quando si scrivono le implementazioni
- **Rendono più facile la comprensione del codice e la sua manutenzione**
- **Possono essere analizzate da tool automatici**

Annotazioni JML

- Per rendere JML facile da usare le annotazioni JML

vengono aggiunte come commenti nel file .java tra

/*@ . . . @*/ , o dopo //@

- Le condizioni sono scritte come espressioni booleane di java con alcuni operatori in più:

\result, \forall, \old, ==>, ...

- e alcune parole chiave

requires, ensures, invariant, ...

Il .java contiene il codice e il suo modello formale secondo DbC

Il .java è un valido file java

Notazione base di JML

JML permette di aggiungere come commenti particolari ad un classe Java:

- precondizioni ai metodi

  //@ requires <espressione booleana>;

- postcondizioni ai metodi

  //@ ensures <espressione booleana>;

- invarianti di classe

  //@ invariant <espressione booleana>;

Contratti incompleti

Nota che un metodo può avere un contratto "debole" non ancora definitivo

Ad esempio il metodo **`debit`** di account:

- richiede che l'ammontare sia positivo
- non garantisce nulla

```
/*@ requires amount >= 0; ensures true;@*/
public int debit(int amount){...}
```

Questa è la postcondizione di default

Altro comandi JML

Espressione	Significato
`a ==> b`	a implica b
`a <== b`	a consegue da b (cioè b implica a)
`a <==> b`	a se e solo se b
`a <=!=> b`	non (a se e solo se b)
`\old(E)`	valore di E prima della chiamata
`\result`	risultato della chiamata del metodo

Esempio con <==>

Nelle espressioni booleane di JML si può usare l'operatore <==> che vuol dire "se e solo se"

A <==> B è (A && B) || (!A && !B)

Ad esempio se ho un metodo `minore` che prende in ingresso j e n e restituisce true se e solo se j< n:

```
//@ ensures \result <==> j < n;
boolean minore(int j, int n) {return j < n;}
```

Quantificatori

Nelle espressioni di JML posso inserire anche quantificatori:

- universali \forall - per ogni elemento
- esistenziali \exists – esiste un elemento
- generali come \sum, \product, \min, \max
- numerici \num_of

La sintassi è

<quant.> <tipo> <var>; <range predicate>; <espressione>

Esempio

Tutti gli studenti contenuti nel Collection `juniors` hanno advisor (dato dal metodo `getAdvisor()`):

```
(\forall Student s; juniors.contains(s);
                    s.getAdvisor() != null)
```

Esempio su quantificatori (1)

Scriviamo il contratto per un metodo che restituisce il minimo di un array

```
public static int  find_min (int a[])
```

PRE: l'array a non è null e ha almeno un elemento

```
requires a != null && a.length >= 1;
```

- se a è vuoto non saprei cosa restituire

POST: l'elemento restituito è effettivamente minore di tutti gli elementi dell'array:

```
ensures (\forall int i; 0 <= i &&
            i <  a.length; \result <= a[i]);
```

- questo non basta: find_min potrebbe restituire MIN_INT

Esempio su quantificatori (2)

La post condizione completa è:

POST: l'elemento restituito è effettivamente minore di tutti gli elementi dell'array e appartiene all'array a

- **esiste un elemento in a che è uguale a result**

```
ensures
(\forall int i; 0 <= i && i <  a.length;
                                \result <= a[i])
&& (\exists int i; 0 <= i && i < a.length;
                                \result == a[i]);
```

non_null

Molti invarianti e pre e post condizioni richiedono che un certo riferimento non sia mai null

L'operatore non_null si usa per esprimere questi requisiti

Esempio

```
public class Directory {
private /*@ non_null @*/ File[] files;
void createSubdir
            (/*@ non_null @*/ String name){…}
```

\old(VARIABILE)

Usata nelle postcondizioni per accedere al valore della variabile prima che il metodo viene eseguito

Esempio

```
public class Contatore {
  integer n;
//@ ensures n == \old(n)+1
public void incrementa() { n ++; }
...
}
```

Esempio: conto in banca

Sia Account una classe che modella un conto in banca [onesta]– senza dbc:

```
public class Account{
    int balance;
    static final int minBalance = 1000
    /** crea un nuovo conto con una
        certa somma iniziale*/
    public Account (int initialAmount){…}
    /** preleva dei soldi */
    public int withdraw(int amount) {…}
    /** deposita un certo importo */
    public int deposit(int amount){…}
```

Contratto per withdraw

	OBBLIGHI	**BENEFICI**
Cliente	(*Soddisfare le precondizioni*:) Assicurarsi che la somma non sia ne' troppo grande ne' troppo piccola.	(*dalle postcondizioni*) Avere il conto aggiornato con la somma ritirata
Fornitore	(*Soddisfare le postcondizioni)* Aggiornare il conto togliendo la somma ritirata	(*Dalle precondizioni*) Calcoli più semplici: può assumere che la somma sia corretta

DBC per withdraw

Quando ritiro un somma amount

PRE: 0<= amount <= balance – minBalance

POST: il bilancio viene diminuito di amount e

il valore restituito dal metodo è uguale al nuovo ammontare sul conto:

- **uso \result per riferirmi al valore restituito**

```
/*@requires amount >= 0
        && amount <= balance - minBalance;
   ensures balance == \old(balance) - amount
        && \result == balance;@*/
public int withdraw(int amount) {
     balance -= amount; return balance;}
```

DBC per nuovo Account

Quando **creo** un account :

PRE - chiedo che `initialAmount` sia maggiore di `minBalance` per poter creare un conto:

- **uso `requires` per le precondizioni**

POST - assicuro che sul conto ci sarà l'intera somma inizialmente versata:

- **uso `ensures` per le postcondizioni**

```
/** crea un nuovo conto */
//@ requires initialAmount >= minBalance;
//@ ensures balance == initialAmount;
public Account (int initialAmount){…}
```

DBC per deposit

Quando **deposito**

PRE: chiedo che la somma depositata sia positiva

POST: assicuro che il bilancio verrà aumentato di quanto ho versato

- **uso old per riferirmi ad un valore prima dell'invocazione di un metodo**

```
//@ requires amount >= 0;
//@ ensures balance == \old(balance)+ amount;
public int deposit(int amount){…}
```

Invariante per Account

In ogni caso

INV: un conto non potrà avere un bilancio inferiore al minimo

- **uso invariant per specificare un invariante**

```
public class Account {
…
/*@ invariant balance >= minBalance; @*/
…
```

Operazione vs dichiarazione

istruzione	postcondizione
`balance = balance - amount`	`ensures balance == \old(balance) - amount`
OPERAZIONE	DICHIARAZIONE
Come?	Cosa?
IMPLEMETAZIONE	SPECIFICA
Comando	Query
Istruzione	Espressione
Imperativo	Controllo

pure

Per dichiarare metodi *pure* ovvero senza *side effect* e quindi usabili nella specifica JML alla stregua di funzioni matematiche

Esempio

```
static /*@ pure @*/ int abs (int x){
 if (x>=0) return x; else return -x;
}
```

Visibilità (1)

JML impone delle regole di visibiltà simili a Java

Ad es.:

```
public class Bag{
  private int n;
  //@ requires n > 0;                    // WRONG !!
  public int extractMin(){ ... }
  ...
}
```

non è corretta, perchè il metodo pubblico extractMin fa riferimento al campo privato n

Visibilità (2)

```
public int pub; private int priv;
//@ requires i <= pub;
public void pub1 (int i) { ... }
//@ requires i <= priv;
private void priv1 (int i) ...
//@ requires i <= pub && i <= priv;   // WRONG !!
public void pub2(int i) { ... }
```

Visibilità: spec_public

Specifica che una variabile privata può essere usata in una specifica pubblica JML

Esempio

```
public class Bag{
private /*@ spec public @*/ int n;
//@ requires n > 0;
public int extractMin(){ ... }
...
}
```

Come alternativa, usare nella specifica i metodi accessori **get...()** dichiarati *pure*

Ereditarietà: also

- La parola chiave **`also`** va usata per i metodi overridden per ereditare la specifica del metodo super

  Vedi metodo toString nell'esempio Person

- le precondizioni della sottoclasse vengono relazionate secondo la logica di un **or** disgiunto con le precondizioni della superclasse

- le postcondizioni della sottoclasse vengono congiunte (**and**) con le postcondizioni della superclasse

- invarianti vengono congiunti (come le postcondizioni)

Esempio

```
public class Person {
   private String name;
   private int weight;
   //Restituisce il nome di una persona
   public String toString(){return nome;}
   //Restituisce il peso di una persona
   public int getWeight (){return weight;}
   //Aumenta il peso di una persona
   public void addKgs(int kgs){
       weight += kgs;}
   //metodo costruttore per creare oggetti Persona
   public Person(String n){
       name = n;
       weight = 0;}}
```

Un esempio completo

```
public class Person {
   private /*@ spec_public non_null @*/ String
name;
   private /*@ spec_public @*/ int weight;

   //@ invariant !name.equals("") && weight >= 0;

   @// also ensures \result != null;
   public String toString() {return name;};

   //@ ensures \result == weight;
   public int getWeight() {return weight;}
```

<<continua nella slide successiva>>

Un esempio completo (Cont.)

```
/*@
    @ requires kgs >= 0;
    @ ensures weight ==\old(weight)+kgs;
    @*/
  public void addKgs(int kgs) {weight +=
kgs;};

  /*@
    @ requires !n.equals("");
    @ ensures n.equals(name) && weight == 0;
    @*/
  public Person(/*@ non_null @*/ String n) {
        name = n;
        weight = 0;} }
```

assert (1)

Per richiedere che una certa condizione sia verificata ad un certo punto all'interno del codice

Esempio

```
if (i <= 0 || j < 0) { ...
} else if (j < 5){
            //@ assert i > 0 && 0 <= j && j < 5;
            ...
} else {
            //@ assert i > 0 && j > 5;
            ...
}
```

assert (2)

- **Nota che anche Java (da 1.4) ha la parola chiave assert (con uguale scopo)**
- **In JML, assert è più espressivo!**

Ad esempio se voglio richiedere che un vettore **a[]** non contenga null…

In Java (con assert):

```
for (n = 0; n < a.length; n++)
                        assert(a[n] != null);
```

In JML:

```
/*@ assert
(\forall int i; 0<=i && i<n ;a[i] != null);@*/
```

in Java però viene riconosciuto e controllato dal compilatore e per attivare il controllo run-time basta usare l'opzione –ea nel comando java

assert e contratti

L'uso di `assert` Java può essere un'alternativa per esprimenre contratti → vedi Lab

esercizi

1. **Definire le pre-condizioni e le post-condizioni di un metodo Java che calcola l'area di una triangolo rettangolo non degenere.**
2. **Definire le pre- e le post-condizioni di un metodo Java che calcola il *massimo* tra due interi positivi.**
3. **Definire un contratto JML per un metodo che calcola il massimo degli elementi di un array.**
4. **Definire un contratto JML per un metodo che calcola la somma degli elementi di un array.**
5. **Scrivere un contratto JML per un metodo booleano che ricerca un valore x intero in un array di interi ordinato in modo crescente.**
6. **Dare la definizione Java di un metodo *insertMax* che inserisce in un array non vuoto di interi un intero x che sia maggiore di tutti gli altri elementi già esistenti nell'array. Definire in JML pre-condizioni e post-condizioni per tale metodo.**

Uso di un contratto per garantire la

CORRETTEZZA DEL SOFTWARE

Uso del contratto

1. **Documentazione**
   - può essere controllata sintatticamente, esportata in formato javadoc, …
2. **Controllo run-time**
   - le condizioni possono essere attivate o disattivate run-time (rallentano l'esecuzione)
     - attivate durante lo sviluppo e durante il testing
     - disattivate sul codice distribuito
3. **Prova di correttezza del software**
   - dimostrare che il contratto è rispettato dall'implementazione

Cenni di Correttezza del Software

La correttezza è una nozione relativa: consistenza dell'implementazione rispetto la sua specifica.

- nel nostro caso il contratto è la specifica

Notazione base:

- A: istruzioni o programma
- P: precondizioni di A
- Q: postcondizioni di A
- correttezza : {P} A {Q} “Hoare triple”

ogni esecuzione di A che inizi in uno stato che soddisfa P termina in uno stato che soddisfi Q

Contratto di SQRT

Sia SQRT una routine per il calcolo della radice quadrata

La correttezza di questa routine richiede la scrittura del contratto:

- A: ... algoritmo per calcolare y come radice quadrata di x (con precisione epsilon)
- P: l'input è positivo

$$\{x >= 0\}$$

- Q: y approssima esattamente il quadrato di x con un errore pari a epsilon

$$\{abs\ (y\ ^\wedge\ 2 - x) <= epsilon\}$$

Esempio di correttezza

A: singola istruzione n := n + 9

P: n > 5

Q: n > 13

Correttezza: {n > 5} n := n + 9 {n > 14}

la dimostrazione di correttezza è complessa e difficilmente effettuabile con strumenti automatici

Correttezza di una Classe

Nel DBC una classe è corretta se sono corretti:

- operazione di creazione

per ogni nuova istanza INV vale

- $\{PRE_{costr}\}$ constructor $\{INV \wedge POST_{costr}\}$

- ogni altro metodo OP

se chiamo un metodo OP con PRE vero (e INV) allora dopo l'esecuzione del metodo vale POST e continua a valere INV

- $\{PRE_{op} \wedge INV\}$ OP $\{POST_{op} \wedge INV\}$